ЗВІТ

про виконання лабораторної роботи №7

«Робота з JSON та підготовка даних для відображення»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Завдання: 1. Створити JSON-файл із початковими даними (мінімум 5 елементів: id, title, isFavorite, status).
2. Додати створений JSON-файл у Flutter-проєкт (у папку assets).
3. Додати шлях до JSON-файлу у файл pubspec.yaml.
4. Реалізувати зчитування JSON-файлу у застосунку та перетворення даних у структури Dart.
5. Створити модель даних (Model class) для опису одного елемента JSON.
6. Реалізувати у моделі методи: fromJson() та toJson().
7. Перетворити JSON-масив у список моделей (List<Model>).
8. Відобразити дані у застосунку за допомогою ListView.builder.
9. Реалізувати додавання нового елемента до списку.
10. Реалізувати редагування існуючого елемента (зміну назви або статусу).
11. Реалізувати видалення елемента зі списку.
12. Реалізувати перемикання стану елемента (додати до обраного, змінити статус).
13. Реалізувати запис оновлених даних у JSON-файл.
14. Перевірити правильність відображення та оновлення даних у застосунку.

1 Написання коду-розв’язку завдання:

```
// === assets/data/tasks.json ===
[
  {
    "id": 1,
    "title": "Оптимізація запитів до бази даних",
    "isFavorite": true,
    "status": "active"
  },
  {
    "id": 2,
    "title": "Рефакторинг архітектури Flutter-екранів",
    "isFavorite": false,
    "status": "done"
  },
  {
    "id": 3,
    "title": "Інтеграція REST API та обробка помилок",
    "isFavorite": false,
    "status": "active"
  },
  {
    "id": 4,
    "title": "Написання віджет-тестів та перевірка стану",
    "isFavorite": true,
    "status": "done"
  },
  {
    "id": 5,
    "title": "Оновлення темної палітри кольорів Material 3",
    "isFavorite": false,
    "status": "active"
  },
```

```
  {
    "id": 6,
    "title": "Підготовка збірки та публікація в Play Store",
    "isFavorite": false,
    "status": "active"
  }
]

// === lib/main.dart ===
import 'package:flutter/material.dart';
import 'screens/lab7_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Менеджер завдань — Лабораторна №7',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFFD4FF00),
          brightness: Brightness.dark,
          primary: const Color(0xFFD4FF00),
          surface: const Color(0xFF15181F),
        ),
        scaffoldBackgroundColor: const Color(0xFF0D0F13),
      ),
      home: const Lab7Screen(),
    );
  }
}

// === lib/models/task_model.dart ===
import 'dart:convert';

/// Перелік можливих статусів завдання (enum)
enum TaskStatus {
  active,
  done,
}

/// Модель завдання (Model class) із 4 обов'язковими типами полів:
/// - int (ідентифікатор)
/// - String (назва завдання)
/// - bool (прапорець обраного)
/// - enum TaskStatus (поточний статус виконання)
class TaskModel {
  final int id;
  String title;
  bool isFavorite;
  TaskStatus status;

  TaskModel({
    required this.id,
    required this.title,
    this.isFavorite = false,
    this.status = TaskStatus.active,
  });

  /// Конструктор для створення об'єкта з JSON (п. 6 ТЗ)
  factory TaskModel.fromJson(Map<String, dynamic> json) {
    return TaskModel(
      id: json['id'] as int,
      title: json['title'] as String,
      isFavorite: json['isFavorite'] as bool? ?? false,
      status: json['status'] == 'done' ? TaskStatus.done : TaskStatus.active,
    );
  }

  /// Метод для серіалізації об'єкта у JSON-мапу (п. 6 ТЗ)
  Map<String, dynamic> toJson() {
    return {
```

```
      'id': id,
      'title': title,
      'isFavorite': isFavorite,
      'status': status.name,
    };
  }

  /// Зручний геттер для перевірки завершеності
  bool get isDone => status == TaskStatus.done;

  /// Метод для створення модифікованої копії
  TaskModel copyWith({
    int? id,
    String? title,
    bool? isFavorite,
    TaskStatus? status,
  }) {
    return TaskModel(
      id: id ?? this.id,
      title: title ?? this.title,
      isFavorite: isFavorite ?? this.isFavorite,
      status: status ?? this.status,
    );
  }
}

/// Перетворення JSON-рядка у список моделей (п. 7 ТЗ)
List<TaskModel> tasksFromJson(String jsonString) {
  final List<dynamic> data = jsonDecode(jsonString) as List<dynamic>;
  return data.map((e) => TaskModel.fromJson(e as Map<String, dynamic>)).toList();
}

/// Серіалізація списку моделей у відформатований JSON-рядок
String tasksToJson(List<TaskModel> tasks) {
  const encoder = JsonEncoder.withIndent('  ');
  return encoder.convert(tasks.map((e) => e.toJson()).toList());
}

// === lib/services/task_storage_service.dart ===
import 'dart:io';
import 'package:flutter/services.dart' show rootBundle;
import 'package:path_provider/path_provider.dart';
import '../models/task_model.dart';

/// Сервіс для роботи з JSON-файлами (зчитування з assets та локальне збереження)
class TaskStorageService {
  static const String assetPath = 'assets/data/tasks.json';
  static const String localFileName = 'tasks.json';

  File? _cachedFile;

  /// 1. Зчитування початкового JSON-файлу з папки assets (п. 4 ТЗ)
  Future<String> readJsonFromAssets() async {
    return await rootBundle.loadString(assetPath);
  }

  /// 2. Отримання шляху до локального файлу в документах застосунку (path_provider)
  Future<File> _getLocalFile() async {
    if (_cachedFile != null) return _cachedFile!;

    if (Platform.environment.containsKey('FLUTTER_TEST')) {
      final tempDir = Directory.systemTemp;
      _cachedFile = File('${tempDir.path}/tasks_test_$pid.json');
      return _cachedFile!;
    }

    final directory = await getApplicationDocumentsDirectory();
    _cachedFile = File('${directory.path}/$localFileName');
    return _cachedFile!;
  }

  /// 3. Зчитування завдань:
  /// Якщо існує локальний файл — читаємо з нього.
  /// Якщо це перший запуск — зчитуємо з assets та копіюємо у локальний файл.
  Future<List<TaskModel>> loadTasks() async {
    try {
      final file = await _getLocalFile();
      if (file.existsSync()) {
        final content = file.readAsStringSync();
```

```
        if (content.trim().isNotEmpty) {
          return tasksFromJson(content);
        }
      }
    } catch (_) {
      // У разі відсутності або помилки переходимо до assets
    }

    final assetJson = await readJsonFromAssets();
    final tasks = tasksFromJson(assetJson);
    await saveTasks(tasks);
    return tasks;
  }

  /// 4. Запис списку моделей у локальний JSON-файл (п. 13 ТЗ)
  Future<void> saveTasks(List<TaskModel> tasks) async {
    try {
      final file = await _getLocalFile();
      final jsonString = tasksToJson(tasks);
      file.writeAsStringSync(jsonString, flush: true);
    } catch (_) {
      // Ігноруємо помилки I/O
    }
  }

  /// 5. Скидання списку до початкового стану з assets
  Future<List<TaskModel>> resetToDefault() async {
    final assetJson = await readJsonFromAssets();
    final tasks = tasksFromJson(assetJson);
    await saveTasks(tasks);
    return tasks;
  }
}

// === lib/screens/lab7_screen.dart (фрагмент) ===
class _Lab7ScreenState extends State<Lab7Screen> {
  final TaskStorageService _storageService = TaskStorageService();

  // Основне джерело даних: список моделей List<TaskModel> (п. 7 ТЗ)
  List<TaskModel> _tasks = [];
  bool _isLoading = true;

  // Фільтр для перегляду
  TaskFilter _currentFilter = TaskFilter.all;
  // ... (пропуск неістотних рядків)
  // 1. Зчитування даних із JSON-файлу (п. 4 ТЗ)
  Future<void> _loadTasksFromJson() async {
    try {
      final loadedTasks = await _storageService.loadTasks();
      if (mounted) {
        setState(() {
          _tasks = loadedTasks;
          _isLoading = false;
        });
      }
    } catch (e, st) {
  // ... (пропуск неістотних рядків)
    }
  }

  // 2. Збереження оновлених даних у JSON-файл (п. 13 ТЗ)
  Future<void> _persistTasks() async {
    await _storageService.saveTasks(_tasks);
  }

  // Обчислення статистики у вигляді Map<String, int>
  Map<String, int> get _statistics => {
    'total': _tasks.length,
    'favorites': _tasks.where((t) => t.isFavorite).length,
    'done': _tasks.where((t) => t.status == TaskStatus.done).length,
    'active': _tasks.where((t) => t.status == TaskStatus.active).length,
  };
  // ... (пропуск неістотних рядків)
  @override
  Widget build(BuildContext context) {
    final filtered = _filteredTasks;

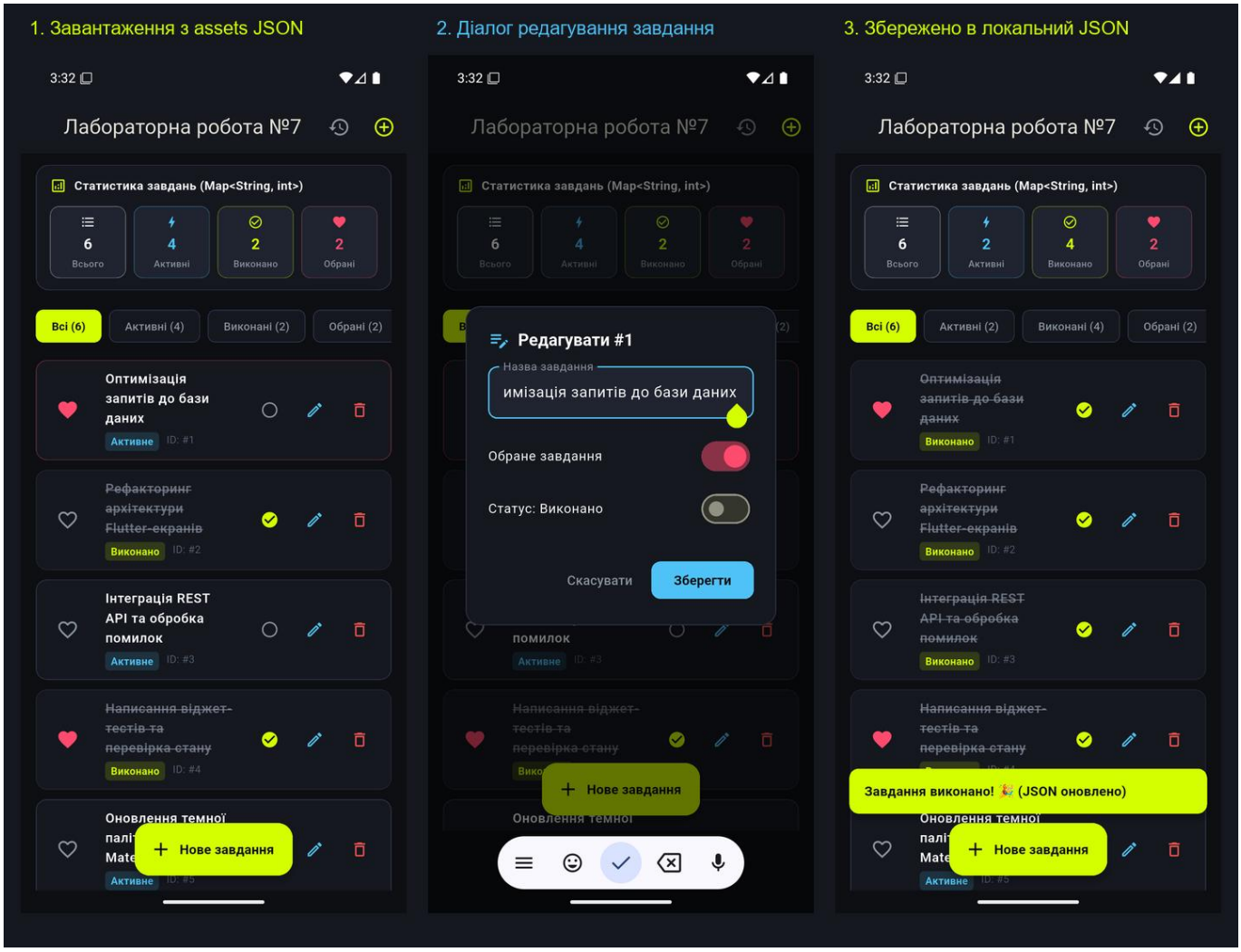
    return Scaffold(
  // ... (пропуск неістотних рядків)
```

```
                    // 1. Статистика у вигляді Map<String, int>
                    StatsSummaryCard(stats: _statistics),
                    const SizedBox(height: 14),

                    // 2. Фільтри завдань
                    Row(
                      children: [
                        _buildFilterChip(
                          'Всі (${_tasks.length})',
                          TaskFilter.all,
                        ),
  // ... (пропуск неістотних рядків)
                      ],
                    ),
                    const SizedBox(height: 12),

                    // 3. Основний список завдань ListView.builder (п. 8 ТЗ)
                    Expanded(
                      child: filtered.isEmpty
  // ... (пропуск неістотних рядків)
                          : ListView.builder(
                              padding: const EdgeInsets.only(bottom: 80),
                              itemCount: filtered.length,
                              itemBuilder: (context, index) {
                                final task = filtered[index];
                                return TaskListItem(
                                  task: task,
  // ... (пропуск неістотних рядків)
                                );
                              },
                            ),
                    ),
  // ... (пропуск неістотних рядків)
  Widget _buildFilterChip(String label, TaskFilter filter) {
    final isSelected = _currentFilter == filter;
    return Expanded(
      child: GestureDetector(
        onTap: () {
          if (_currentFilter != filter) {
            setState(() {
              _currentFilter = filter;
            });
          }
        },
  // ... (пропуск неістотних рядків)
      ),
    );
  }
}
```

2 Результат виконання коду:



1. Що таке JSON та для чого він використовується у програмуванні? JSON (JavaScript Object Notation) — це легкозважений текстовий формат представлення та обміну структурованими даними. Завдяки незалежності від платформи й мови програмування, простоті синтаксису та легкій людиночитній формі, JSON є загальноприйнятим стандартом для взаємодії клієнтських мобільних застосунків із веб-серверами через REST API, а також для збереження локальних конфігурацій і кешу на пристрої.

2. Які основні елементи структури JSON? Основу синтаксису JSON складають два типи структур: об'єкти та масиви. Об'єкт — це невпорядкований набір пар «ключ : значення», який записується у фігурних дужках { }, де ключ завжди є рядком у подвійних лапках. Масив — це впорядкована послідовність значень (елементів), яка записується у квадратних дужках [ ], де елементи відокремлюються комами.

3. Які типи даних можуть використовуватися у JSON? У форматі JSON підтримуються такі базові типи значень: рядок (String, обов'язково у подвійних лапках "..."), число (Number, цілі або з плаваючою крапкою), логічне значення (Boolean: true або false), порожнє значення (null), масив (Array: [...]) та вкладений об'єкт (Object: { ... }).

4. Яку бібліотеку використовують у Dart для роботи з JSON? Для роботи з форматом JSON у мові Dart використовується стандартна бібліотека dart:convert (import

'dart:convert';). Вона надає готові утиліти для серіалізації та десеріалізації: функції jsonDecode() та jsonEncode(), а також класи JsonDecoder і JsonEncoder.
5. Яка різниця між функціями jsonDecode() та jsonEncode()? Функція jsonDecode() приймає текстовий рядок у форматі JSON (String) та виконує парсинг, перетворюючи його на відповідні структури даних Dart (Map або List). Навпаки, функція jsonEncode() виконує серіалізацію — перетворює об'єкти Dart (наприклад, Map, List чи примітивні типи) на текстовий рядок формату JSON.
6. У який тип даних перетворюється JSON-об’єкт після декодування? Після виконання функції jsonDecode() кореневий або вкладений JSON-об'єкт { } перетворюється у мові Dart на асоціативний масив типу Map<String, dynamic>, де ключами є рядки, а значеннями — динамічні сутності, тип яких визначається за вмістом кожного поля.
7. У який тип даних перетворюється JSON-масив після декодування? Після виклику jsonDecode() кореневий або вкладений JSON-масив [ ] перетворюється у мові Dart на список динамічних елементів типу List<dynamic>. Надалі кожен елемент цього списку зазвичай кастується до Map<String, dynamic> або обробляється фабричним конструктором моделі.
8. Як зчитати JSON-файл з папки assets у Flutter? Для зчитування вбудованого ресурсу з каталогу assets використовують клас rootBundle із пакету flutter/services.dart: final String jsonString = await rootBundle.loadString('assets/data/tasks.json');. Для коректної роботи шлях до файлу або каталогу обов'язково попередньо декларується у секції flutter -> assets файлу конфігурації pubspec.yaml.
9. Як записати JSON-дані у файл на пристрої? Запис здійснюється за допомогою класу File із бібліотеки dart:io: final file = File(filePath); await file.writeAsString(jsonString);. Попередньо шлях до каталогу файлів застосунку (наприклад, Documents directory) отримують за допомогою функції getApplicationDocumentsDirectory() із плагіна path_provider, після чого перетворюють колекцію моделей у JSON-рядок через jsonEncode() і зберігають його на диску.
10. Для чого використовується пакет path_provider? Пакет path_provider надає уніфікований міжплатформний доступ до стандартних системних шляхів збереження файлів на мобільних і десктопних операційних системах (Android, iOS, Windows, macOS, Linux). Він дозволяє без прив'язки до специфіки конкретної ОС отримати шлях до каталогу документів користувача (getApplicationDocumentsDirectory()), каталогу тимчасових файлів кешу (getTemporaryDirectory()) тощо.
11. Яке призначення методу fromJson()? Метод fromJson() (найчастіше реалізований як фабричний конструктор factory Model.fromJson(Map<String, dynamic> json)) призначений для створення типізованого об'єкта моделі на основі десеріалізованої з JSON мапи. Він виконує безпечне перетворення типів, встановлює значення за

замовчуванням та конвертує примітивні типи (наприклад, String) у спеціалізовані сутності (enum або DateTime).

12. Яке призначення методу toJson()? Метод toJson() виконує зворотну операцію серіалізації: він перетворює екземпляр моделі Dart у мапу Map<String, dynamic>. У цій мапі всі складні типи (наприклад, enum чи Date) переводяться у стандартні типи, зрозумілі формату JSON (рядки, числа, логічні значення), що дає змогу передати мапу у функцію jsonEncode().

13. Як перетворити JSON-масив у список моделей? Спочатку вхідний JSON-рядок декодується за допомогою jsonDecode() у список dynamic-елементів (List<dynamic>). Потім за допомогою методу .map() кожен елемент списку (який є Map<String, dynamic>) передається у фабричний конструктор моделі fromJson() і результат приводиться до типізованого списку через .toList(): final List<TaskModel> items = (jsonDecode(jsonString) as List).map((e) => TaskModel.fromJson(e)).toList();.

14. Які операції можна виконувати з даними, що зберігаються у JSON-файлі? Із даними у JSON-файлі можна виконувати повний набір операцій CRUD: Create (додавання нових записів із подальшою перезаписами файлу), Read (зчитування та парсинг файлу під час завантаження екрана), Update (редагування властивостей існуючих записів, наприклад, зміна назви, прапорця обраного чи статусу enum) та Delete (видалення запису зі списку та збереження актуального стану колекції у файл).

Висновок: На цій лабораторній роботі, я опанував практичні навички роботи з форматом JSON у мові Dart та фреймворку Flutter, створив початковий файл assets/data/tasks.json та зареєстрував його в pubspec.yaml, реалізував типізовану модель даних TaskModel із методами серіалізації fromJson() та toJson(), побудував сервіс збереження даних TaskStorageService з використанням бібліотеки path_provider та пакету dart:convert, реалізував відображення динамічного списку через ListView.builder та ListTile, додав операції створення, редагування, вилучення завдань та перемикання станів із миттєвою синхронізацією у локальний JSON-файл, а також перевірив коректність роботи бізнес-логіки та інтерфейсу за допомогою автоматизованих тестів та на емуляторі Android.